

Training Automated Test Oracles to Identify Semantic Bugs

Charaka Geethal

Faculty of Information Technology

Monash University

Melbourne, Australia

charaka.kapugamawasangamagedon@monash.edu

Abstract—Can a machine find and fix a *Semantic Bug*? A *Semantic Bug* is a deviation from the expected program behaviour that causes to produce incorrect outputs for certain inputs. To identify this category of bugs, the knowledge on the expected program behaviour is essential. The reason is that a program with a semantic bug does not fail (i.e., crash or hang) in the middle of the execution in most scenarios. Thus, only a human (a user or a developer) knowing the correct program behaviour can detect this kind of bug by observing the output. However, identifying bugs solely through human effort is not practical for all software. A *Test Oracle* is any procedure used to differentiate the correct and incorrect behaviours of a program. This dissertation mainly focuses on developing learning techniques to produce *Automated Test Oracles* for programs with semantic bugs. Also, discovering methods to incorporate human knowledge effectively for the learning techniques is another concern. The automated test oracles could make semantic bug detection more efficient. Also, such test oracles could guide *Automated Program Repair* tools to generate more accurate fixes for semantic bugs.

I. INTRODUCTION

In *Software Engineering*, delivering fault-free software is a significant concern. However, bugs are introduced to software due to various facts. Under one category of bugs, the program produces incorrect, unexpected outputs for certain inputs. These bugs are called **Semantic Bugs** or **Functional Bugs**. Although the output for certain inputs are wrong, the program having a semantic bug does not fail (crash or hang [1]) immediately in most scenarios. Thus, given an input, comparing the program output with the expected program output is the only method of detecting this category of bugs. Therefore, the expected correct program behaviour should be known in the detection process. For this reason, semantic bugs are considered as *application-specific* [2].

As an example, assume that there is a program to calculate the addition of two (2) numbers. Due to a semantic bug, this program returns that $2 + 2 = 5$, which is incorrect. In this scenario, to identify the output as incorrect, it is essential to know the expected correct output. The expected output is $2 + 2 = 4$, as the program has been written to add two numbers. Without this knowledge, detecting this bug would be impossible.

The nature of *Semantic Bugs* described above has created many challenges in testing and fixing those. Due to the necessity of expected program behaviour, only the human (the developer or the user) can detect this category of bugs.

Usually, a semantic bug is reported with one failing input, i.e., an input triggering the bug. However, only the human can answer about further failing inputs. All these facts have made semantic bug detection a challenging task. Nevertheless, several recent studies emphasize the necessity of detecting and fixing this category of bugs. The findings of the works Lin et al. [2], Arash et al. [3] and Wan et al. [4] indicate that semantic bugs can occur in different types of software systems, and those can lead to many adverse consequences (e.g. security vulnerabilities). Therefore, detecting and fixing this category of bugs is essential, overcoming the difficulties mentioned above.

In software testing, given a System Under Test (SUT), the challenge of distinguishing the correct, expected behaviour from the incorrect behaviour is called the **Oracle Problem** [5]. Based on this definition, a **Test Oracle** can be defined as any procedure used to differentiate the correct and the incorrect behaviour of the *System Under Test*. Furthermore, a test oracle determines whether a test case *passing* or *failing*. A program is non-testable without a test oracle [6].

As the knowledge on expected program behaviour is essential, the only possible test oracle for semantic bugs is the *Human Oracle*. However, testing with human oracles is inefficient and impractical for many software. Thus, techniques to develop automated semantic bug test oracles should be discovered. The survey paper of Earl et al. [5] shows that the topic of *Test Oracle Automation* has received a less research attention compared to the other topics in *Automated Software Testing*. Therefore, exploring techniques to automate semantic bug test oracles could help to fill a significant research gap.

This dissertation focuses on developing learning techniques to train automated test oracles for semantic bugs. Such test oracles could make efficient semantic bug detection. Also, those test oracles could be used in *Automated Program Repair* [7] to produce more accurate fixes for semantic bugs.

The expected contributions of this dissertation are as follows.

- 1) We will present a comprehensive analysis on the existing works on test oracle automation. Here, we will highlight the limitations of the existing techniques in applying those to semantic bugs.
- 2) We will introduce test oracle learning techniques for different kinds of program inputs to address semantic

bugs in a wide range of programs.

- 3) We will introduce some effective methods in our learning techniques to train highly accurate test oracles under a fixed number of human queries.

II. PROBLEM DEFINITION

The main focus of this dissertation is to develop *learning techniques* to train automated test oracles for semantic bugs. Moreover, we explore methods to systematically incorporate the human knowledge about the semantic bug into the learning techniques. These automated test oracles could be applied in software testing and *Automated Program Repair* [7].

We conduct our exploration based on the following research questions.

- RQ.1.** How do we systematically involve the human in deriving an automated oracle?
- RQ.2.** How do we design a classification technique that can predict test failures for arbitrary inputs?
- RQ.3.** How do we design an efficient learning technique that maximizes the accuracy of the classifier given a limited number of queries to the human?

As described in section I, a semantic bug is usually reported with a single failing test case exposing it. However one failing input is not enough to train an automated test oracle for the bug. Therefore, we need more labelled test cases. The human is the most accessible source regarding a semantic bug. Thus, we have to use test cases labelled as *passing* and *failing* by the human to train test oracles. However, such human involvement cannot be expected limitlessly, as it is impractical and has many disadvantages. Therefore, in **RQ.1.**, we explore methods to get the human systematically involved in the oracle learning techniques. Through these methods, the oracle learning techniques will request the label of a test case only when necessary in training.

From machine learning perspective, a test oracle performs a classification task distinguishing the *passing* and *failing* test cases of the *System Under Test(SUT)*. As semantic bugs can be arisen in different types of software, we explore applicable classification algorithms for different kinds of program inputs (e.g. numbers, strings) under **RQ.2.** This approach helps to maintain the generalizability of this study. Also, we develop techniques to train automated test oracles with minimal failing inputs. The reason is that finding failing inputs for some semantic bugs can be difficult with human effort.

As one failing test case is not enough to train automated test oracles, we must generate more test cases to be labelled by the human. Assuming that the human can label only a limited number of test cases, **RQ.3.** focuses on obtaining the maximum use of the available labelling queries. Furthermore, we explore techniques to determine what should be labelled from the generated such that the test oracle's accuracy increases, using the queries. Also, finding possible ways to generate test cases that improve the accuracy when labelled by the human is another concern.

The learning techniques developed through these research questions will train an automated test oracle for a semantic

bug, beginning from just one failing test case. Here, those will systematically interact with the human to label the test cases generated and selected by themselves.

III. RESEARCH METHODOLOGY

We parallelly explore the answer to the research questions described in section II. Literature Reviews and experiments are the main research methods used in this work.

A. Literature Review

The focus on each research question of the literature review is as follows.

1) **RQ.1.** *Systematic Involvement of Human Knowledge:*

In training automated test oracles, we expect to get the human involved in labelling test cases used in training. The existing works on *Active Learning* [8] provide useful insights to perform this kind of task systematically. Thus, our literature review focuses on *Active Learning* techniques.

2) **RQ.2.** *Developing Learning Techniques for Different Types of Program Inputs:* Many studies have been done on *classification algorithms* [9] in machine learning. As the automated test oracles are developed as classifiers, we focus on different classification techniques in our literature analysis. Regarding a classification technique, we consider the types of inputs that the techniques can handle. Also, we consider the ability to handle the *Class Imbalance Problem* [10], as the total number of failing inputs is significantly less than the passing inputs in some semantic bugs. We conduct preliminary experiments to assess this property.

3) **RQ.3.** *Improving Accuracy Under Limited Human Queries:* We explore automated test generation methods that can produce test cases advantageous for test oracle training. Also, we focus on different machine learning setups that can guide these automated test generation methods (e.g. Adversarial Machine Learning [11], Q-Learning [12] etc.)

B. Developing Oracle Learning Techniques

Our objective is to learn an automated test oracle from a single failing test case of a bug. In developing an oracle learning technique, we select a classification algorithm, considering a set of program input types, addressing **RQ.2.** Then, we combine a test generation method and an active learning technique with that (Fig. 1).

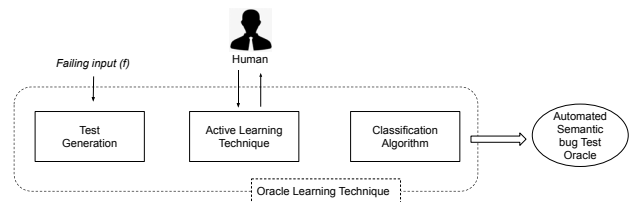


Fig. 1. Components of an oracle learning technique

The test generation method generates new test cases from the given failing input. Some test generation methods can produce test cases advantageous for training, complying with the objectives of **RQ.3.** If this does not happen, we apply some techniques to guide the test generation process to produce this type of test cases (e.g. Adversarial Machine Learning [11], Q-Learning [12] etc.).

We use active learning methods to select test cases from the generated to be presented to the human for labelling. These techniques can do this selection based on the current state of the classifier being trained [8], which avoids unnecessary labelling. Thus, we employ these techniques to involve the human systematically to the training process (**RQ.1.**).

C. Dataset Selection

To evaluate the learning techniques, we use *Program Repair Benchmarks* (e.g. IntroClass [13], Quixbugs [14], Codeflaws [15] etc). We select benchmarks for our experiments based on the following criteria.

- 1) There should be programs with real world defects leading to semantic bugs.
- 2) For each faulty subject (program), there should be a version where the bug has been fixed, i.e., *golden version*. The golden version indicates the expected, correct program behaviour.
- 3) For each subject, there should be a manually created, labelled test suite, in which there should be at least one failing test case.

D. Experimental Setup & Evaluation

We select the buggy subjects taking the inputs that the learning technique supports from the benchmarks. To simulate the human in the experiments, given an input, we compare the output of the buggy version with its golden version. If both the outputs are equal, it is a *Passing* input, otherwise a *Failing* input (Fig. 2).

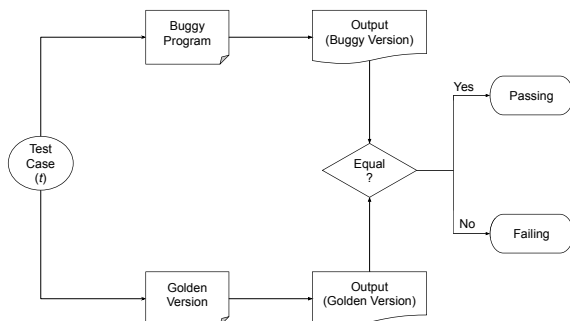


Fig. 2. Simulation of Human Oracle

We begin the experiment by selecting a failing test case randomly from the labelled test suite. After the learning technique generates an automated oracle, we use it to predict the labels of the test cases in the labelled test suite. Then we measure the following:

- 1) *Accuracy* : Percentage of correctly predicted test cases from the labelled test suite.
- 2) *Recall for failing inputs - Conditional Accuracy* : Percentage of correctly predicted failing test cases from the failing test cases in the labelled test suite.
- 3) *Human labelling effort* : Number of test cases that the human had to label in training.

The reason to measure the recall for failing inputs is to assess the ability of the learning technique to handle the *Class Imbalance Problem* [10]. To minimize the impact of randomness and to gain statistical power for the experimental results, we repeat the experiment 30 times per subject and get the average of the above measures. The accuracy and the conditional accuracy indicate how successfully the learning technique addresses **RQ.2.** Meanwhile, the labelling effort shows how well it contributes to **RQ.1.** and **RQ.3.**

Additional evaluation metrics might be required depending on the learning techniques. We will introduce them accordingly.

IV. CURRENT PROGRESS

So far, we have made some contributions to the research questions presented in Section II. Related to **RQ.1.**, we introduced a method to select test cases with a higher failure probability for human labelling. The *Active Learning* approach proposed by Houlb et al. [16] is the base for our test case selection method. Additionally, we expect to analyze the applicability of *Bayesian* [17] approaches to select test cases for human labelling.

Under **RQ.2.**, we found that the technique *Incremental SMT Constraint Learner(INCAL)* [18] can be used to learn an *SMT (Satisfiability Modulo Theory)* [19] formula from *numeric* examples. Such an SMT formula can be used as a classifier. Thus, we used INCAL [18] to derive test oracles for programs taking numeric inputs. Also, to handle *string* inputs, we applied Regular Positive and Negative Inference (RPNI) [20] algorithm. This algorithm can infer a *Deterministic Finite Automata (DFA)* [21] explaining the pattern of a given set of strings.

Related to **RQ.3.**, we explored how *Mutational Fuzzing* [22] can be applied to generate test cases that are advantageous for oracle training. In *numeric* inputs, *Mutational Fuzzing* can explore the neighbourhood of the given failing input. Labelling the neighbourhood inputs and using those in training can generate more accurate test oracles. Also, we found that *Delta Debugging Minimization(ddmin)* [23] can be applied to generate test inputs that are advantageous in grammar inference for the failing string inputs of a bug.

We have developed two techniques to generate semantic bug test oracles for programs taking numeric inputs and string inputs, respectively. Each technique takes only one failing

input of the *System Under Test* (SUT) and interacts with the human systematically in oracle training.

A. LEARN2FIX

LEARN2FIX [24] is a technique to learn test oracles for programs taking numeric inputs. It takes with one failing input of the buggy program and returns an SMT formula [19] satisfied only by the failing inputs of the program.

LEARN2FIX [24] uses the technique *Incremental SMT Constraint Learner* [18] to learn an SMT formula from the test cases. Also, it follows the active learning methodology influenced by the work of Houlb et al. [16] to involve the human systematically in oracle training. To generate more failing inputs from the given failing input, LEARN2FIX uses *Mutational Fuzzing* [22].

We selected the programs taking *numeric* inputs and containing semantic bugs from the benchmark Codeflaws [15] and conducted some experiments on those to evaluate LEARN2FIX.

The experimental results show that the automated test oracles produced by LEARN2FIX can classify test cases with 63% - 80% accuracy. Also, these oracles can predict failing test cases with at least 78% accuracy in most subjects.

To achieve this accuracy, LEARN2FIX has to label less than half of the generated test cases.

B. Minimization Based Active Failure Grammar Inference

To deal with string inputs, we developed an algorithm to infer a grammar describing the pattern of the failing inputs of a bug. Similar to LEARN2FIX [24], this algorithm begins with one failing input of the bug. Finally, it returns a collection of *Deterministic Finite Automata* (DFA) [21] describing the pattern of the failing inputs.

This grammar inference algorithm is based on the following algorithms.

- 1) Delta Debugging Minimization (*ddmin*) [23]
- 2) Regular Positive and Negative Inference (RPNI) [20]

ddmin [23] is an algorithm that can trace the *minimal failing input* reproducing the bug from a given failing input. It uses the divide and conquer approach for this task. Unlike numeric inputs, a failing string input can have parts that are not related to the failure. *ddmin* can remove such parts from the string. As the divide and conquer approach is used, some new passing and failing inputs are generated during the minimization process.

Given a set of positive and a set of negative examples, Regular Positive and Negative Inference (RPNI) algorithm can develop a Deterministic Finite Automata (DFA) [21] that accepts all the positive examples and reject all the negative examples. Applying RPNI [20] to the test cases generated in *ddmin*, the initial DFA is developed. This DFA is subjected to several generalization steps afterward.

To evaluate this method, we conducted some experiments on the programs taking string inputs and containing semantic

bugs in the benchmarks IntroClass [13], Quixbugs [14], and Codeflaws [15].

The overall accuracy for most subjects in the benchmarks is more than 75%. Also, the accuracy of predicting failing inputs by our technique is more than 70% for the majority of subjects.

V. RELATED WORK

Semantic Bugs or *Functional Bugs* are a critical challenge in software testing. The reason is that the only possible oracle for this type of bug is the human (the developer or the user). The study of Lin et al. [2] shows that the developer's lack of understanding of the system and its requirements is the main reason for introducing semantic bugs. The findings of the studies Lin et al. [2], Arash et al. [3], and Wan et al. [4] show that semantic bugs can lead to many issues, such as security vulnerabilities, in various software systems. Lin's study [2] further reveals that semantic bugs can increase as the software evolves. Sometimes, this category of bugs can remain hidden for a longer period (e.g. semantic bugs in open source blockchain systems [4]). All these facts emphasize the necessity of finding and fixing semantic bugs.

In software testing, the *Test Oracle* is an essential component. According to Weyuker's study [6], a program is non-testable without a test oracle. The survey papers of Earl et al. [5], Mauro et al. [25], Paulo et al. [26] and Rafael et al. [27] discuss different aspects of the *Oracle Problem*. Especially, Earl's study presents a classification for test oracles, which is helpful to guide this work. Moreover, Earl's study [5] highlights the less research attention on automation of test oracles in *Automated Software Testing*. Regarding semantic bugs, this is a critical issue, as human oracles are not practical for all types of software.

Machine Learning has been applied to develop automated test oracles. The approaches of Hu Jin et al. [28], Vanmali et al. [29], and Shahamiri et al. [30] use *Artificial Neural Networks* to develop test oracles. These test oracles are suitable for semantic bugs. However, many practical problems associated with *training* test oracles for semantic bugs have not been addressed in these approaches. For example, the *Class Imbalance Problem*, i.e., very few failing inputs and more passing inputs, has not been handled in these *Artificial Neural Network* based methods. To produce an accurate test oracle, these methods require a balanced dataset. However, a semantic bug is usually reported with one input for which the program produces an incorrect output.

To summarize, detecting and fixing *Semantic Bugs* are essential in software engineering. However, developing automated test oracles for this purpose has received a less research attention. Also, the practical problems associated with semantic bugs have not been solved in the existing machine learning based solutions. Therefore, the methods to develop automated semantic bug test oracles should be explored.

REFERENCES

- [1] Z. Li, L. Tan, X. Wang, S. Lu, Y. Zhou, and C. Zhai, "Have things changed now?: an empirical study of bug characteristics in modern open source software," in *Proceedings of the 1st workshop on Architectural and system support for improving software dependability*. ACM, 2006, pp. 25–33.
- [2] L. Tan, C. Liu, Z. Li, X. Wang, Y. Zhou, and C. Zhai, "Bug characteristics in open source software," *Empirical Software Engineering*, vol. 19, no. 6, pp. 1665–1705, 2014.
- [3] A. Vahabzadeh, A. M. Fard, and A. Mesbah, "An empirical study of bugs in test code," in *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Sep. 2015, pp. 101–110.
- [4] Z. Wan, D. Lo, X. Xia, and L. Cai, "Bug characteristics in blockchain systems: a large-scale empirical study," in *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*. IEEE, 2017, pp. 413–424.
- [5] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE transactions on software engineering*, vol. 41, no. 5, pp. 507–525, 2014.
- [6] E. J. Weyuker, "On testing non-testable programs," *The Computer Journal*, vol. 25, no. 4, pp. 465–470, 1982.
- [7] L. Gazzola, D. Micucci, and L. Mariani, "Automatic software repair: A survey," *IEEE Transactions on Software Engineering*, vol. 45, no. 1, pp. 34–67, 2017.
- [8] B. Settles, "Active learning literature survey," University of Wisconsin-Madison Department of Computer Sciences, Tech. Rep., 2009.
- [9] C. Zhang, C. Liu, X. Zhang, and G. Alpanidis, "An up-to-date comparison of state-of-the-art classification algorithms," *Expert Systems with Applications*, vol. 82, pp. 128–150, 2017.
- [10] R. Longadge and S. Dongre, "Class imbalance problem in data mining review," *arXiv preprint arXiv:1305.1707*, 2013.
- [11] V. Kuleshov and D. Precup, "Algorithms for multi-armed bandit problems," *arXiv preprint arXiv:1402.6028*, 2014.
- [12] C. J. Watkins and P. Dayan, "Q-learning," *Machine learning*, vol. 8, no. 3–4, pp. 279–292, 1992.
- [13] C. Le Goues, N. Holtschulte, E. K. Smith, Y. Brun, P. Devanbu, S. Forrest, and W. Weimer, "The manybugs and introclass benchmarks for automated repair of c programs," *IEEE Transactions on Software Engineering*, vol. 41, no. 12, pp. 1236–1256, 2015.
- [14] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "Quixbugs: A multi-lingual program repair benchmark set based on the quixey challenge," in *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, 2017, pp. 55–56.
- [15] S. H. Tan, J. Yi, Yulis, S. Mehtaev, and A. Roychoudhury, "Codeflaws: A programming competition benchmark for evaluating automated program repair tools," in *2017 IEEE/ACM 39th International Conference on Software Engineering Companion (ICSE-C)*, 2017, pp. 180–182.
- [16] A. Holub, P. Perona, and M. C. Burl, "Entropy-based active learning for object recognition," *2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops*, pp. 1–8, 2008.
- [17] I. Rish *et al.*, "An empirical study of the naive bayes classifier," in *IJCAI 2001 workshop on empirical methods in artificial intelligence*, vol. 3, no. 22, 2001, pp. 41–46.
- [18] S. Kolb, S. Teso, A. Passerini, and L. De Raedt, "Learning smt (lra) constraints using smt solvers," in *IJCAI*, vol. 18, 2018, pp. 2333–2340.
- [19] C. Barrett and C. Tinelli, "Satisfiability modulo theories," in *Handbook of Model Checking*. Springer, 2018, pp. 305–343.
- [20] C. De la Higuera, *Grammatical inference: learning automata and grammars*. Cambridge University Press, 2010.
- [21] M. V. Lawson, *Finite automata*. CRC Press, 2003.
- [22] C. Chen, B. Cui, J. Ma, R. Wu, J. Guo, and W. Liu, "A systematic review of fuzzing techniques," *Computers & Security*, vol. 75, pp. 118–137, 2018.
- [23] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.
- [24] M. Böhme, C. Geethal, and V.-T. Pham, "Human-in-the-loop automatic program repair," in *Proceedings of the 2020 IEEE International Conference on Software Testing, Verification and Validation*, ser. ICST 2020, 2020, pp. 1–12.
- [25] M. Pezze and C. Zhang, "Automated test oracles: A survey," in *Advances in computers*. Elsevier, 2014, vol. 95, pp. 1–48.
- [26] P. A. Nardi and E. F. Damasceno, "A survey on test oracles," pp. 50–59, 2015.
- [27] R. A. Oliveira, U. Kanewala, and P. A. Nardi, "Automated test oracles: State of the art, taxonomies, and trends," in *Advances in computers*. Elsevier, 2014, vol. 95, pp. 113–199.
- [28] H. Jin, Y. Wang, N.-W. Chen, Z.-J. Gou, and S. Wang, "Artificial neural network for automatic test oracles generation," in *2008 International Conference on Computer Science and Software Engineering*, vol. 2. IEEE, 2008, pp. 727–730.
- [29] M. Vanmali, M. Last, and A. Kandel, "Using a neural network in the software testing process," *International Journal of Intelligent Systems*, vol. 17, no. 1, pp. 45–62, 2002.
- [30] S. R. Shahamiri, W. M. Wan-Kadir, S. Ibrahim, and S. Z. Hashim, "Artificial neural networks as multi-networks automated test oracle," *Automated Software Engg.*, vol. 19, no. 3, p. 303–334, Sep. 2012. [Online]. Available: <https://doi.org/10.1007/s10515-011-0094-z>